Bulkhead is a **resilience and fault-isolation pattern** used in distributed systems and microservices to prevent one overloaded or failing component from consuming resources needed by other components.

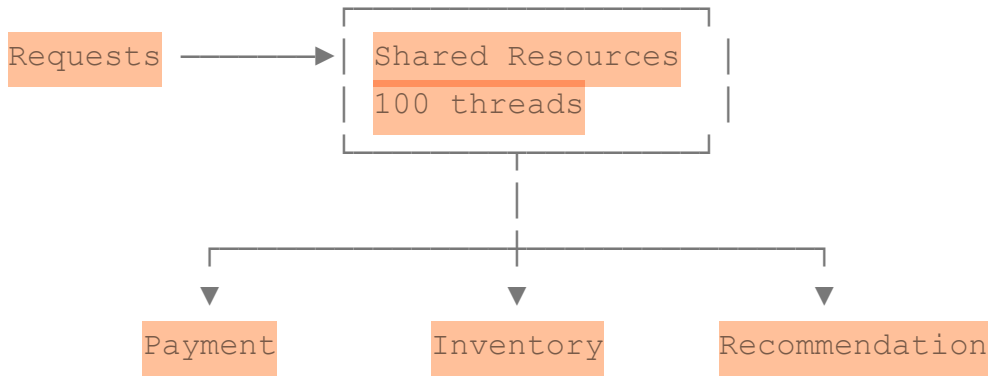
The name comes from **ship bulkheads**: a ship is divided into watertight compartments so that if one compartment floods, the entire ship does not sink.

Simple example

Suppose an e-commerce application has:

- Order Service
- Payment Service
- Inventory Service
- Recommendation Service

Without a bulkhead, all requests might share the same thread pool:



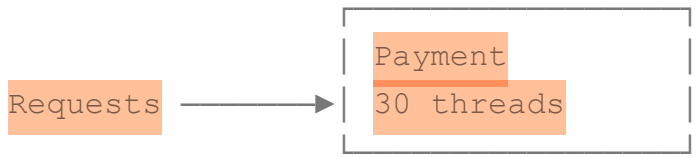
If the **Recommendation Service** becomes extremely slow, requests waiting for recommendations can consume all 100 threads.

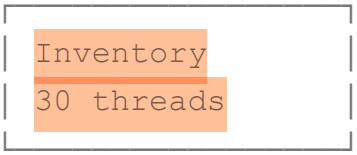
Now even **Payment** and **Inventory** requests cannot get threads.

The failure has propagated.

With Bulkhead

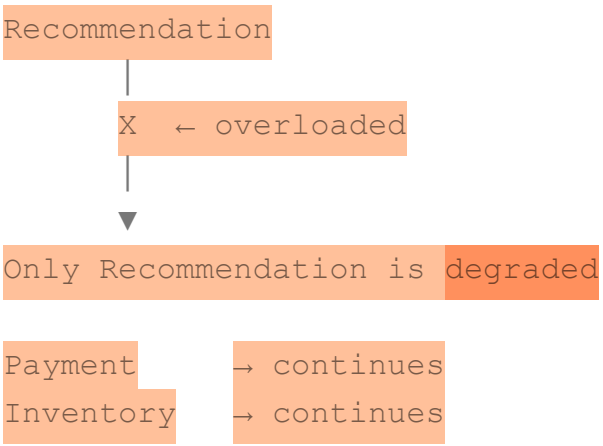
We isolate resources:







If Recommendation becomes unhealthy:



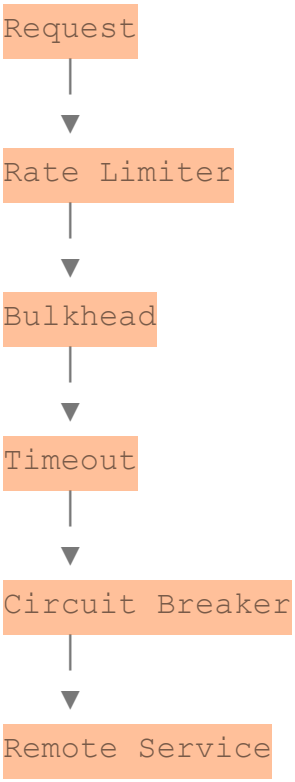
That is the fundamental idea of **Bulkhead Isolation**.

Bulkhead vs Circuit Breaker

These patterns are often used together, but they solve different problems.

Pattern	Protects against	Main mechanism
Timeout	Slow dependency	Stop waiting after a limit
Circuit Breaker	Repeated dependency failures	Stop sending requests temporarily
Bulkhead	Resource exhaustion	Isolate/constrain resources
Retry	Transient failures	Try again
Rate Limiting	Excessive request volume	Limit incoming requests

A typical resilient call might therefore look like:



The exact ordering depends on the architecture and framework.

Two common types of Bulkhead

1. Thread-pool / execution isolation

Give different dependencies separate execution pools.

For example:

```
Payment         → 20 threads
Inventory       → 20 threads
Shipping        → 10 threads
Recommendations → 10 threads
```

If Shipping becomes slow, it cannot consume Payment's threads.

This was particularly common with frameworks such as **Netflix Hystrix**.

---

2. Semaphore isolation

Instead of allocating separate thread pools, limit the **number of concurrent executions**.

For example:

```
Payment API
Maximum concurrent requests = 20
```

Request #21 is rejected or handled through fallback rather than being allowed to consume unlimited resources.

This is often more lightweight than thread-pool isolation.

Bulkhead in a Spring Boot application

A common modern choice is **Resilience4j**.

Conceptually:

```
@Bulkhead(
    name = "paymentService",
    type = Bulkhead.Type.SEMAPHORE
)
public PaymentResponse pay(PaymentRequest request) {
    return paymentClient.pay(request);
}
```

You might configure something like:

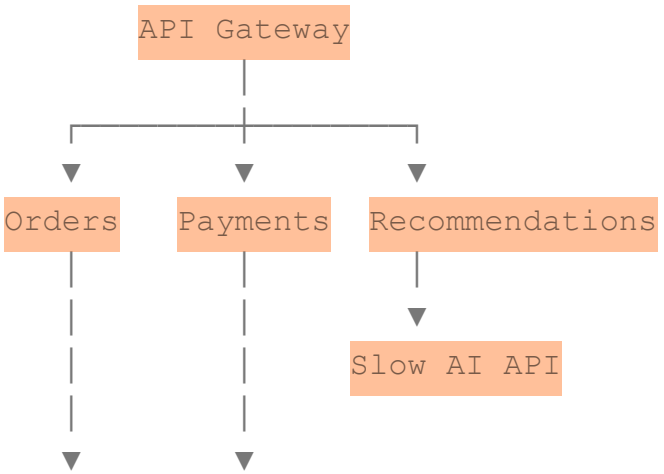
```
resilience4j:
  bulkhead:
    instances:
      paymentService:
        maxConcurrentCalls: 20
        maxWaitDuration: 100ms
```

Meaning:

> At most 20 calls to the payment operation can execute concurrently, and callers won't wait indefinitely for capacity.

Why Bulkhead is important in microservices

Consider this scenario:



Suppose the AI API takes **30 seconds** to respond.

Without bulkheads:

```
10 requests
    ↓
10 threads occupied

100 requests
    ↓
100 threads occupied

More requests
    ↓
Thread pool exhausted
    ↓
Entire application becomes unhealthy
```

With a bulkhead:

```
Recommendation
    │
    ├── max 10 concurrent calls
    │
    └── additional requests → fallback/rejection

Payment
    │
    └── independent capacity
```

The recommendation problem is **contained**.

Important architectural insight

Bulkhead is not simply:

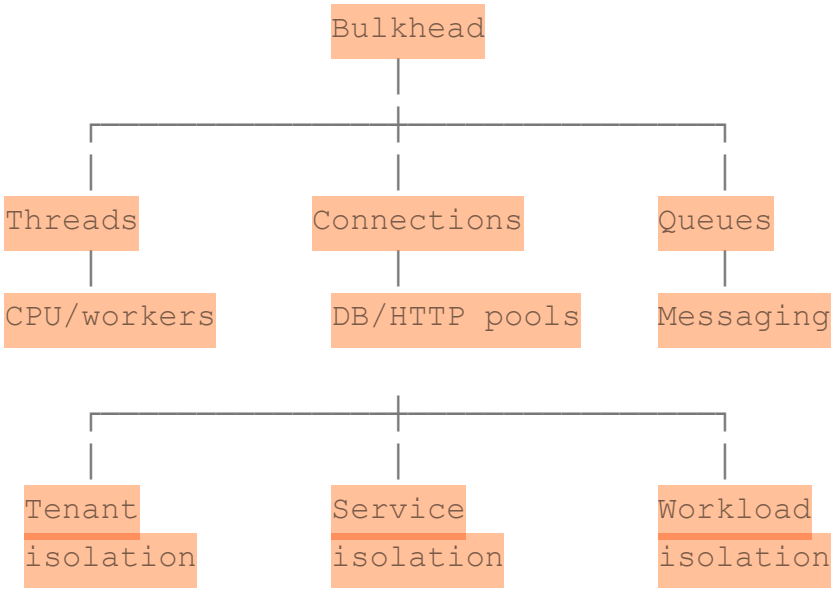
> "Limit the number of requests."

It is:

> **Prevent one workload, dependency, tenant, or subsystem from exhausting resources required by other workloads.**

This distinction becomes especially important in **high-scale distributed systems**.

You can implement bulkheads at several levels:



For **MAANG+/Staff Engineer system-design interviews**, Bulkhead is a particularly important resilience pattern because it demonstrates that you understand **resource isolation, failure containment, backpressure, graceful degradation, and cascading-failure prevention** rather than just individual microservice failures.